WEEK 4 · OBSERVABILITY & SRE

OpsDesk

A telecom observability platform that names the failing layer before anyone joins the call

Presenter

Sebastien Ngembou

Platform

AWS EKS · Terraform · GitHub Actions

github.com/ngems1/observability-sre

A telecom operator does not lack dashboards

During an outage it has too many — and each team's dashboard says their own layer is fine.

`00:00`

Users report failures. Someone is paged.

`00:05`

Network, platform and database teams join a bridge call.

`00:40`

Someone finally works out **which layer** broke.

`00:45`

The fix itself takes five minutes.

A 99.5% monthly target allows about **three and a half hours** of downtime. Spend forty minutes of it deciding whose problem this is, and the diagnosis becomes the outage.

THE CLAIM

Every layer gets a signal only that layer can move.

So the incident ticket names the failing layer before anyone joins a call — and then I broke the platform four different ways to check the name it gives is the right one.

Five layers, five questions

Layer	The question only it answers	The signal
Kubernetes	Are the processes running?	ready vs wanted replicas · restarts
Application	Is the code erroring or burning time?	5xx ÷ requests · request p95 – db p95
Queue	Is work piling up faster than it drains?	SQS visible depth · seconds since poll
Database	Is SQL slow, or refused?	statement p99 · error kind
Network	Can we open a connection at all?	connect_timeout · dns · refused

These are independent, so one fault answers **its** question badly and leaves the rest normal. The pattern across all five is a fingerprint — and no two faults share one.

Evaluated continuously, every 15 seconds — not looked up during an incident

The idea underneath: subtraction

```
request time = app's own work + database time + waiting on dependencies
```

I measure request p95 and database p95. The difference is everything the application did by itself. That is arithmetic, not opinion — which is why one panel separates three different faults:

`both lines rise together`

Database

The app is waiting on storage.

`request rises, database flat`

Application

The time went into our own code.

`database reports no data`

Network

No statement ever completed.

One cluster, two environments, no stored credentials

GitHub Actions — OIDC

CI on every pull request · Release builds an immutable SHA-tagged image, scans it, deploys dev, waits for approval, deploys prod

Terraform — bootstrap · infra · platform

VPC, EKS, RDS, SQS, KMS, IAM, controllers and observability, one module per component

IRSA — per service account

Pods exchange a short-lived OIDC token for temporary AWS credentials. No access key exists anywhere in the system.

EKS "opsdesk" — 3 × m5.large

opsdesk-dev
api · worker

opsdesk-prod
api · worker

observability
Prometheus · Alertmanager · Grafana · Tempo · OTel

kube-system · logging
ALB controller · external-dns · autoscaler · Fluent Bit → CloudWatch

Per environment: its own RDS, SQS queue and DLQ, secrets and IAM roles. Isolated by namespace, ResourceQuota and NetworkPolicy.

Sharing the cluster is the single biggest cost saving — the isolation controls are what make it defensible

LIVE — 4 MINUTES

The application, running

1

OpsDesk on HTTPS

Create a ticket, approve an access request. Real ALB, real certificate, DNS kept by external-dns.

2

Service overview

What the on-call sees when nothing is wrong: SLOs, requests, errors, latency — all green.

3

Break it

Trigger the drill from GitHub Actions, then keep talking while the fault develops.

The environment is destroyed and rebuilt from code every day. What you are looking at was created this morning by a workflow, from an empty account state.

Causes beat symptoms

Most monitoring fails one of two ways: too few alerts and you miss outages, too many and everyone ignores them. Typing the alerts sidesteps the trade-off.

`alert_type = cause`

Names one layer

WorkerDown · DependencyUnreachable · QueueBacklog · DatabaseSlow · DbPoolExhausted

`alert_type = symptom`

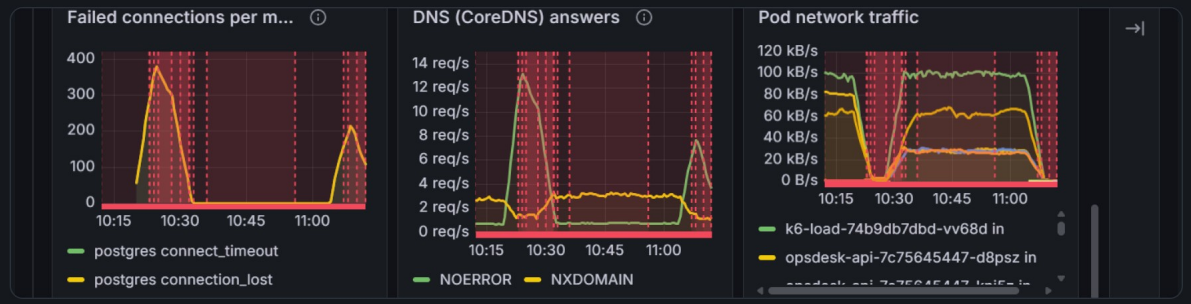
Users are affected

ErrorBudgetBurn · ApiLatencyHigh — real impact, layer unknown

While any cause is firing, Alertmanager **mutes every symptom** in that namespace — and a cause mutes its own consequence. One fault yields one ticket naming the layer, not six tickets describing its echoes.

DRILL 6 — EGRESS TO POSTGRESQL BLOCKED

Every symptom said "database"



`Network row`

connect_timeout climbs from a flat zero to **380/min**

`Database row`

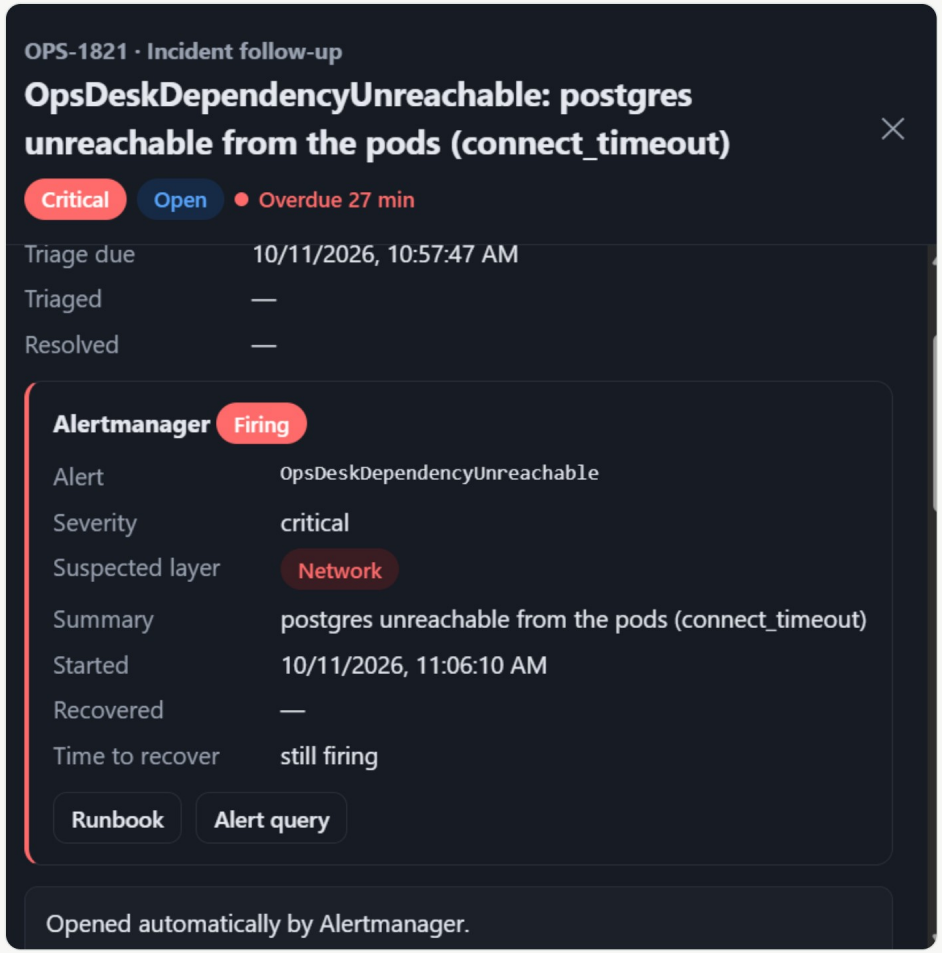
No data. Not a high number — none at all

`RDS console`

CPU normal, instance healthy throughout

Queries failed. Pods went unhealthy. The load balancer returned 503. Any dashboard built on averages would have turned the database tile red — and sent an engineer to the wrong team.

A slow database has a high statement time. An unreachable one has none at all.



The platform said network

Opened automatically by Alertmanager two minutes after the fault. Suspected layer, priority from severity, team from labels, runbook link attached. **No human judgement between the fault and this ticket.**

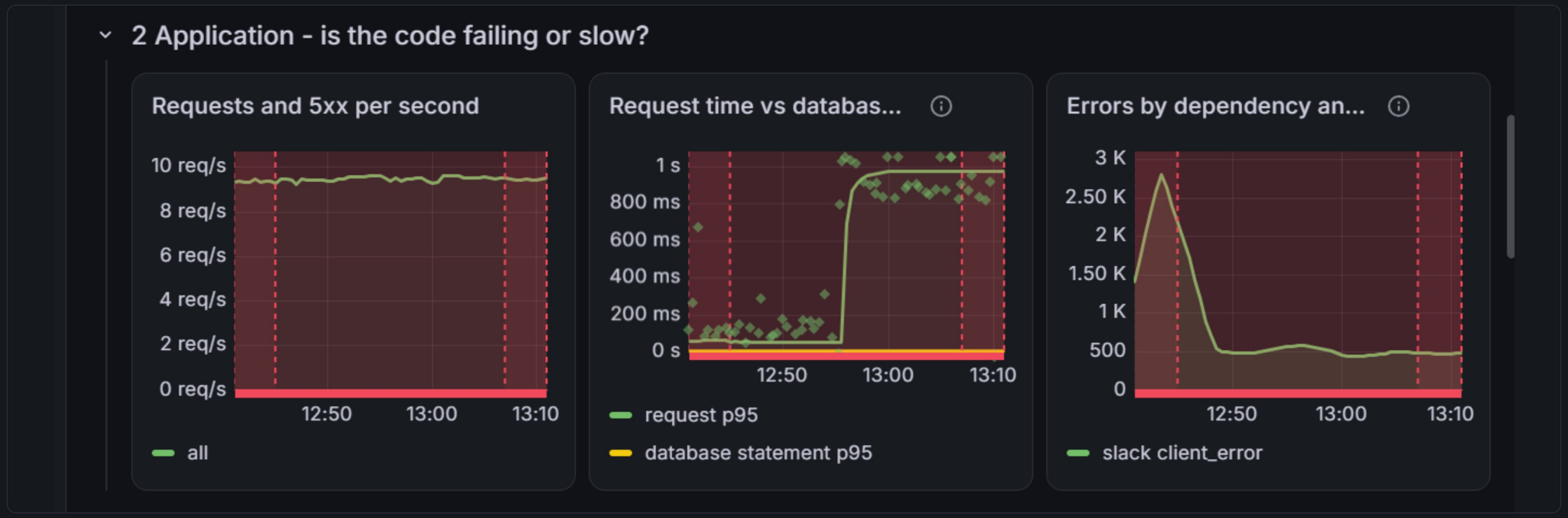
`recovery, measured by the platform`

Started 15:06:10Z · Recovered 15:23:40Z · **17 min 30 s**, computed and written to the ticket with nobody typing a number.

That duration is how long I left the fault running. The figure that measures the platform is the **two minutes** from fault to a correctly-labelled ticket — set by the alert window, not by anyone being awake.

DRILL 2 — 800 MS INJECTED INTO THE APPLICATION

Slow is not the same as broken



`request p95`

100 ms → **1 s**

`database p95`

Does not move

`errors`

Zero. 9 req/s throughout

`verdict`

~900 ms in **our own code**

Honest gap: the Application tile measures error ratio, so a latency-only fault lights no tile — the panel found it, and a p95 term is on the next-steps list

Three alerts true, one ticket at a time

UTC	What happened	Result
16:18	ticket-worker scaled to zero	fault injected
16:20	OpsDeskWorkerDown fires	OPS-13674 · layer kubernetes
16:20	OpsDeskQueueBacklog also true	suppressed — WorkerDown inhibits it
16:39	worker restored, WorkerDown resolves	inhibition lifts
16:40	OpsDeskQueueBacklog now free	OPS-15858 · layer queue
17:06	OpsDeskApiLatencyHigh fires	suppressed — a cause is active

The worker stopping is the cause; the backlog is what a stopped worker **does**. Two tickets would mean two teams investigating one fault. Cause before consequence, three minutes apart, each naming its own layer.

Measured from a live cluster · inhibition rules in terraform/platform/observability.tf

No long-lived AWS credentials exist in this system

identity

GitHub OIDC + IRSA

CI presents a signed token and assumes a role. Pods do the same through their service account. Same mechanism, two places — nothing stored, nothing to rotate, nothing to leak.

least privilege

Scoped per workload

external-dns reaches one hosted zone, not *. Fluent Bit writes only the app log groups. Each environment's roles are trusted only in its own namespace, so dev cannot read prod's secrets.

in the pipeline

Scanning gates the release

Trivy on dependencies and image, Checkov on Terraform and rendered Helm, kubeconform, helm lint — on every pull request. Fixable critical and high findings block the deploy.

Also in place: KMS encryption on secrets, RDS and logs · secrets from Secrets Manager via External Secrets, never in the chart · NetworkPolicy default-deny with an explicit egress allowlist · GuardDuty, Security Hub, Inspector and CloudTrail · TLS 1.3 at the load balancer with HTTP redirected

Findings, owners and remediation: docs/security/findings.md